JAVA 2

Module LAM

Lambdas et références de méthodes

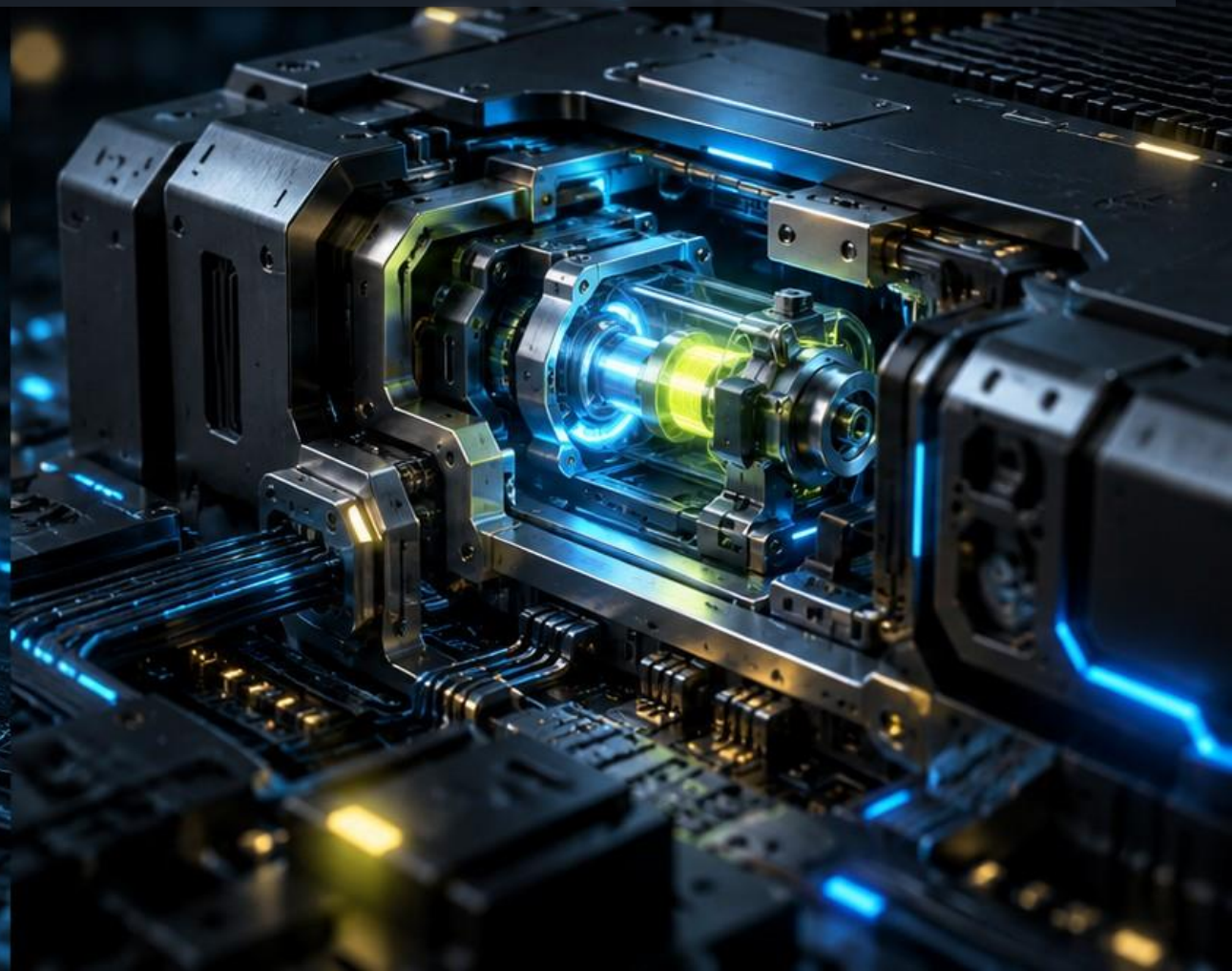
Frédéric Ferrant – 2026-2027

Au programme

1. La classe anonyme
2. La lambda
3. JDK et références

1. Une classe qui perd son nom





Vous passez déjà des comportements

La méthode `removeIf` reçoit un objet, mais elle n'en attend qu'une méthode

Toute la classe existe pour contenir une seule ligne

```java
public class TitreLong implements Predicate<String> {

    private static final int LONGUEUR_MAX = 20;

    @Override
    public boolean test(String titre) {
        return titre.length() > LONGUEUR_MAX;    // la seule ligne utile
    }
}
```

```java
titres.removeIf(new TitreLong());
```

La classe anonyme

Une classe écrite à l'endroit même où l'on crée son objet

Elle n'a pas de nom : elle ne servira qu'ici

Le mot `new` est suivi de l'interface (ou d'une classe abstraite), puis du corps de la classe

```
titres.removeIf(new Predicate<String>() {
    @Override
    public boolean test(String titre) {
        return titre.length() > LONGUEUR_MAX;
    }
});
```

Démonstration



Démonstration

Ce qu'une lambda peut remplacer, et ce qu'elle ne peut pas

Déroulé : Demo_LAM

2. Ce que le compilateur sait déjà



La lambda : ce qui reste

Il y a une écriture encore plus courte que la classe anonyme : on retire tout ce que le compilateur peut déduire

Il reste le paramètre, une flèche, et ce qu'on en fait

```
titres.removeIf(new Predicate<String>() {     // impose par removeIf
    @Override
    public boolean test(String titre) {       // la seule methode a ecrire
        return titre.length() > LONGUEUR_MAX;
    }
});
```

```
titres.removeIf( titre -> titre.length() > LONGUEUR_MAX );
```

Rappel : l'interface fonctionnelle

Une interface fonctionnelle n'a qu'une seule méthode abstraite

C'est celle que la lambda écrit : aucune hésitation possible

`@FunctionalInterface` demande au compilateur de vérifier la règle

```java
@FunctionalInterface
public interface Predicate<T> {      // java.util.function

    boolean test(T t);               // la seule methode abstraite

    // ... plus des methodes default, qui ne comptent pas
}
```

Le type de la lambda vient du contexte

Une lambda n'a pas de type à elle : elle prend celui de sa destination

Une variable, un paramètre : c'est le type cible

Ce type doit être une interface fonctionnelle, sinon quelle méthode écrire ?

```
// la variable fixe le type
Predicate<String> estLong = titre -> titre.length() > LONGUEUR_MAX;
```

```
// le parametre fixe le type : Executable, une seule methode, execute()
assertThrows(IllegalArgumentException.class, () -> salle.reserver(0));
```

assertThrows(Class<T> expectedType, Executable executable)

Asserts that execution of the supplied executable throws an exception of the expectedType and returns the exception.

org.junit.jupiter.api.function

Interface Executable

Functional Interface:

This is a functional interface and can therefore be used as the assignment target for a lambda expression or method reference.

Les formes d'une lambda

Aucun paramètre : des parenthèses vides

Deux paramètres : entre parenthèses, séparés par une virgule

Plusieurs instructions : un bloc, avec son `return`

```
Runnable saluer = () -> System.out.println("Bonjour");
```

```
Comparator<String> parLongueur = (a, b) -> a.length() - b.length();
```

```
Predicate<String> estLong = titre -> {
    String sansEspaces = titre.strip();
    return sansEspaces.length() > LONGUEUR_MAX;
};
```

Exercices



LAM.1 Un tableau de comportements

fascicule LAM

LAM.2 Écrire son interface fonctionnelle

fascicule LAM

Déplier une lambda : IDE.15 ; nommer ce qui n'a pas de nom : CC-MET.1

3. Réutiliser ce qui existe



Les interfaces de `java.util.function`

Interface	Méthode	Elle sait
`Predicate<T>`	`boolean test(T)`	répondre oui ou non
`Function<T, R>`	`R apply(T)`	transformer un `T` en `R`
`Consumer<T>`	`void accept(T)`	faire quelque chose d'un `T`
`Supplier<T>`	`T get()`	fournir un `T`

Les méthodes de COL, sans classe

Au module `COL`, `removeIf` et `computeIfAbsent` recevaient un objet de `TitreLong` ou de `NouvelleListe`, deux classes écrites pour l'occasion

Une lambda les remplace, en une ligne, là où elle sert

La méthode `forEach` reçoit un `Consumer` : elle agit sur chaque élément

```
titres.removeIf(titre -> titre.length() > LONGUEUR_MAX);
```

```
titresParAuteur.computeIfAbsent("Zola", auteur -> new ArrayList<>())
               .add("Germinal");
```

```
titres.forEach(titre -> System.out.println(titre));
```

La référence de méthode

Si la lambda ne fait qu'appeler une méthode, on désigne la méthode

La classe, deux deux-points, le nom (sans parenthèses)

Le paramètre vient de `removeIf`, qui passe chaque titre à `estLong`

```
public class Titres {
    private static final int LONGUEUR_MAX = 20;
    public static boolean estLong(String titre) {
        return titre.length() > LONGUEUR_MAX;
    }
}
```

```
titres.removeIf(titre -> Titres.estLong(titre));
titres.removeIf(Titres::estLong);
```

Un stream, en passant

Un stream fait passer les éléments d'une collection par une suite d'étapes

`filter` garde certains éléments, `map` les transforme, `collect` rassemble le résultat

La collection d'origine n'est pas modifiée

```
String longs = titres.stream()                              // 1. ouvrir le flux
        .filter(titre -> titre.length() > LONGUEUR_MAX)      // 2. garder les longs
        .map(titre -> titre.toUpperCase())                   // 3. mettre en majuscules
        .collect(Collectors.joining(", "));                  // 4. joindre en un texte
```

Le stream, étape par étape

Suivons quatre titres à travers les étapes, avec `LONGUEUR_MAX = 8`.

```
String longs = titres.stream()
    .filter(titre -> titre.length() > LONGUEUR_MAX)
    .map(titre -> titre.toUpperCase())
    .collect(Collectors.joining(", "));
```

Étape	Ce qui en sort
1. `stream`	Nana · La Débâcle · Germinal · Le Docteur Pascal
2. `filter`	La Débâcle · Le Docteur Pascal
3. `map`	LA DÉBÂCLE · LE DOCTEUR PASCAL
4. `collect`	"LA DÉBÂCLE, LE DOCTEUR PASCAL"

Exercices



LAM.3 Celle du JDK, et les références de méthodes

fascicule LAM

Nommer un critère : `CC-NOM.1` ; une seule chose à la fois : `CC-MET.3`

Vous les retrouverez au module FX

Un bouton reçoit le comportement à déclencher au clic

Le type cible est `EventHandler`, une interface fonctionnelle

Le module FX dira le reste

```
bouton.setOnAction(evenement -> afficherTotal());
```

Le fil rouge



G.32 Une fabrique sans `switch`

fil rouge GameHub

Relire G.17 : une stratégie est un comportement